

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No: 5

Roll no: 57 PRN No:12410559 Name of Student: Avni Selote

Subject Name : Deep Learning

Problem Statement : Implement and compare RNN, LSTM, and GRU models for sequence classification, and analyze their performance using appropriate evaluation metrics.

Algorithm :

1. Load the IMDb movie review dataset.
2. Clean the review text and remove unnecessary characters.
3. Split the dataset into training, validation, and testing sets.
4. Tokenize the reviews and convert words into integer sequences.
5. Pad/trim all sequences to a fixed length of 200.
6. Create RNN, LSTM, and GRU models using the same architecture.
7. Train all three models on the training data.
8. Evaluate the models using accuracy, precision, recall, F1-score, and confusion matrix.
9. Compare their performance and training time.
10. Identify the best-performing model for sentiment classification.

About the dataset :

The IMDb Movie Reviews dataset contains 50,000 movie reviews collected from IMDb and is used for binary sentiment classification. Each review is labeled as either positive or negative, with approximately 25,000 reviews in each class. The dataset has two main columns: review, which contains the text of the movie review, and sentiment, which contains its corresponding label. This dataset is suitable for comparing RNN, LSTM, and GRU models for sentiment analysis.

Code & Output :

```
# Import required libraries

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# TensorFlow and Keras
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Embedding,
    SimpleRNN,
    LSTM,
    GRU,
    Dense,
    Dropout
)

# Evaluation metrics
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

# Other utilities
import re
import time
import random

print("TensorFlow version:", tf.__version__)
```

*** TensorFlow version: 2.20.0

```
from google.colab import files
```

```
uploaded = files.upload()
```

Choose Files IMDB Dataset.csv
IMDB Dataset.csv(text/csv) - 66212309 bytes, last modified: 9/11/2026 - 100% done
Saving IMDB Dataset.csv to IMDB Dataset (1).csv

```
# Load the CSV file

df = pd.read_csv("IMDB Dataset.csv")

# Display dataset shape
print("Dataset shape:", df.shape)

# Display first 5 rows
df.head()
```

Dataset shape: (50000, 2)

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

```

# Count positive and negative reviews

sentiment_counts = df["sentiment"].value_counts()

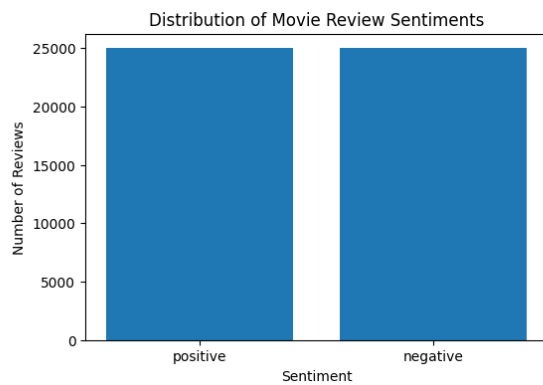
plt.figure(figsize=(6, 4))

plt.bar(
    sentiment_counts.index,
    sentiment_counts.values
)

plt.title("Distribution of Movie Review Sentiments")
plt.xlabel("Sentiment")
plt.ylabel("Number of Reviews")

plt.show()

```



```

def clean_text(text):

    # Convert to lowercase
    text = text.lower()

    # Remove HTML tags
    text = re.sub(r"<br\s*/?>", " ", text)

    # Remove special characters and numbers
    text = re.sub(r"[^a-z\s]", " ", text)

    # Remove multiple spaces
    text = re.sub(r"\s+", " ", text).strip()

    return text

# Apply cleaning function to every review

df["clean_review"] = df["review"].apply(clean_text)

# Compare original and cleaned review

print("Original review:")
print(df["review"].iloc[0])

print("\nCleaned review:")
print(df["clean_review"].iloc[0])

```

```

Original review:
One of the other reviewers has mentioned that after watching just 1 Oz episode you'll be hooked.

Cleaned review:
one of the other reviewers has mentioned that after watching just oz episode you ll be hooked th

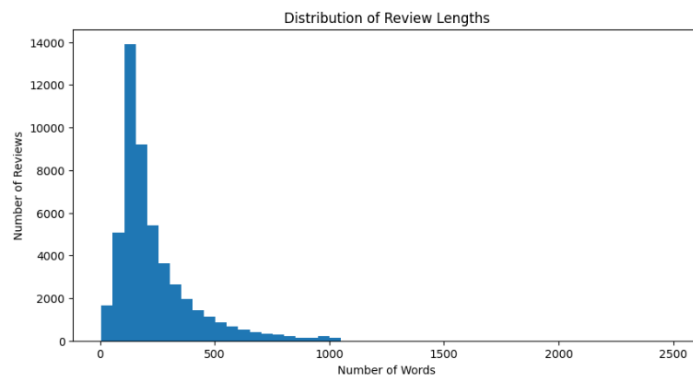
```

```
plt.figure(figsize=(10, 5))

plt.hist(
    df["review_length"],
    bins=50
)

plt.title("Distribution of Review Lengths")
plt.xlabel("Number of Words")
plt.ylabel("Number of Reviews")

plt.show()
```



```
# First split:
# 80% training
# 20% temporary data

train_df, temp_df = train_test_split(
    df,
    test_size=0.20,
    random_state=SEED,
    stratify=df["label"]
)

# Split temporary data equally:
# 10% validation
# 10% testing

val_df, test_df = train_test_split(
    temp_df,
    test_size=0.50,
    random_state=SEED,
    stratify=temp_df["label"]
)

print("Training samples:", len(train_df))
print("Validation samples:", len(val_df))
print("Testing samples:", len(test_df))
```

```
Training samples: 40000
Validation samples: 5000
Testing samples: 5000
```

```
# Pad or truncate sequences to MAX_SEQUENCE_LENGTH
```

```
X_train = pad_sequences(  
    X_train,  
    maxlen=MAX_SEQUENCE_LENGTH,  
    padding="post",  
    truncating="post"  
)  
  
X_val = pad_sequences(  
    X_val,  
    maxlen=MAX_SEQUENCE_LENGTH,  
    padding="post",  
    truncating="post"  
)  
  
X_test = pad_sequences(  
    X_test,  
    maxlen=MAX_SEQUENCE_LENGTH,  
    padding="post",  
    truncating="post"  
)  
  
print("X_train shape:", X_train.shape)  
print("X_val shape:", X_val.shape)  
print("X_test shape:", X_test.shape)
```

```
X_train shape: (40000, 200)  
X_val shape: (5000, 200)  
X_test shape: (5000, 200)
```

```
▶ # Maximum number of words that will be kept in our vocabulary
```

```
MAX_VOCAB_SIZE = 20000
```

```
# Maximum number of words in each review
```

```
MAX_SEQUENCE_LENGTH = 200
```

```
# Size of word embeddings
```

```
EMBEDDING_DIM = 128
```

```
# Batch size for training
```

```
BATCH_SIZE = 64
```

```
print("Maximum vocabulary size:", MAX_VOCAB_SIZE)  
print("Maximum sequence length:", MAX_SEQUENCE_LENGTH)  
print("Embedding dimension:", EMBEDDING_DIM)  
print("Batch size:", BATCH_SIZE)
```

```
... Maximum vocabulary size: 20000  
Maximum sequence length: 200  
Embedding dimension: 128  
Batch size: 64
```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input, Embedding, SimpleRNN, LSTM, GRU, Dense, Dropout

def build_model(model_type):

    model = Sequential([
        Input(shape=(MAX_SEQUENCE_LENGTH,)),

        Embedding(
            input_dim=MAX_VOCAB_SIZE,
            output_dim=EMBEDDING_DIM,
            mask_zero=True
        )
    ])

    if model_type == "RNN":
        model.add(SimpleRNN(128))

    elif model_type == "LSTM":
        model.add(LSTM(128))

    elif model_type == "GRU":
        model.add(GRU(128))

    else:
        raise ValueError("Invalid model type")

    model.add(Dropout(0.3))
    model.add(Dense(64, activation="relu"))
    model.add(Dense(1, activation="sigmoid"))

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model

```

```

# Build RNN
rnn_model = build_model("RNN")

# Build LSTM
lstm_model = build_model("LSTM")

# Build GRU
gru_model = build_model("GRU")

```

```

print("RNN MODEL")
rnn_model.summary()

print("\nLSTM MODEL")
lstm_model.summary()

print("\nGRU MODEL")
gru_model.summary()

```

RNN MODEL
Model: "sequential_3"

Layer (type)	Output Shape	Param #
embedding_3 (Embedding)	(None, 200, 128)	2,560,000
simple_rnn_1 (SimpleRNN)	(None, 128)	32,896
dropout_3 (Dropout)	(None, 128)	0
dense_6 (Dense)	(None, 64)	8,256
dense_7 (Dense)	(None, 1)	65

Total params: 2,601,217 (9.92 MB)
Trainable params: 2,601,217 (9.92 MB)
Non-trainable params: 0 (0.00 B)

... LSTM MODEL
Model: "sequential_4"

Layer (type)	Output Shape	Param #
embedding_4 (Embedding)	(None, 200, 128)	2,560,000
lstm_1 (LSTM)	(None, 128)	131,584
dropout_4 (Dropout)	(None, 128)	0
dense_8 (Dense)	(None, 64)	8,256
dense_9 (Dense)	(None, 1)	65

Total params: 2,699,985 (10.30 MB)
Trainable params: 2,699,985 (10.30 MB)
Non-trainable params: 0 (0.00 B)

GRU MODEL
Model: "sequential_5"

Layer (type)	Output Shape	Param #
embedding_5 (Embedding)	(None, 200, 128)	2,560,000
gru_1 (GRU)	(None, 128)	99,072
dropout_5 (Dropout)	(None, 128)	0
dense_10 (Dense)	(None, 64)	8,256
dense_11 (Dense)	(None, 1)	65

Total params: 2,667,393 (10.18 MB)
Trainable params: 2,667,393 (10.18 MB)
Non-trainable params: 0 (0.00 B)

```
print("-----")
print("Training RNN")
print("-----")

start_time = time.time()

rnn_history = rnn_model.fit(
    X_train,
    y_train,

    validation_data=(
        X_val,
        y_val
    ),

    epochs=EPOCHS,

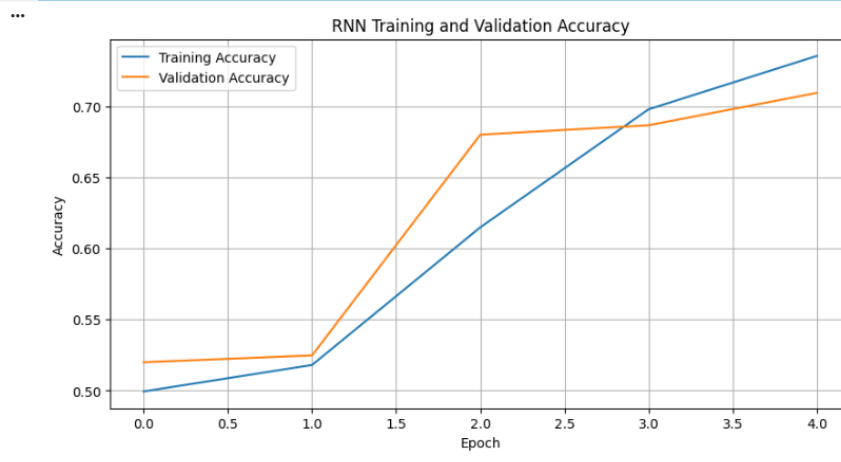
    batch_size=BATCH_SIZE,

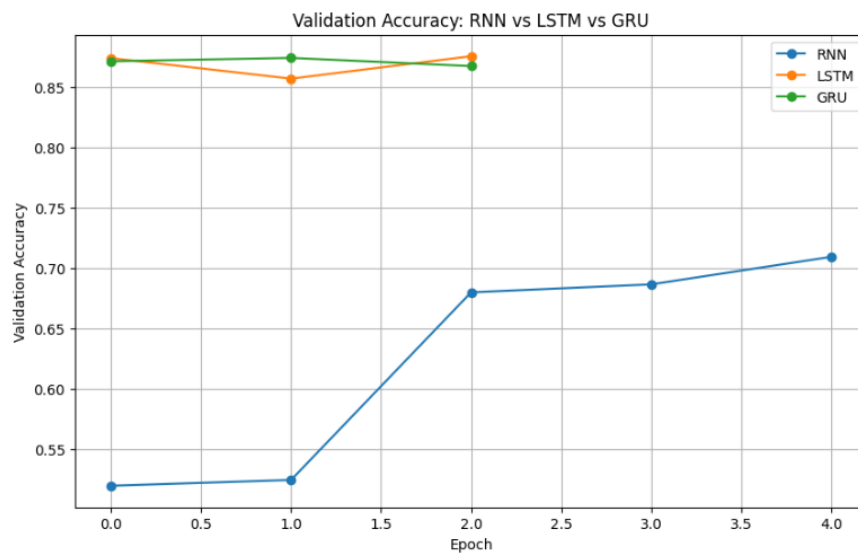
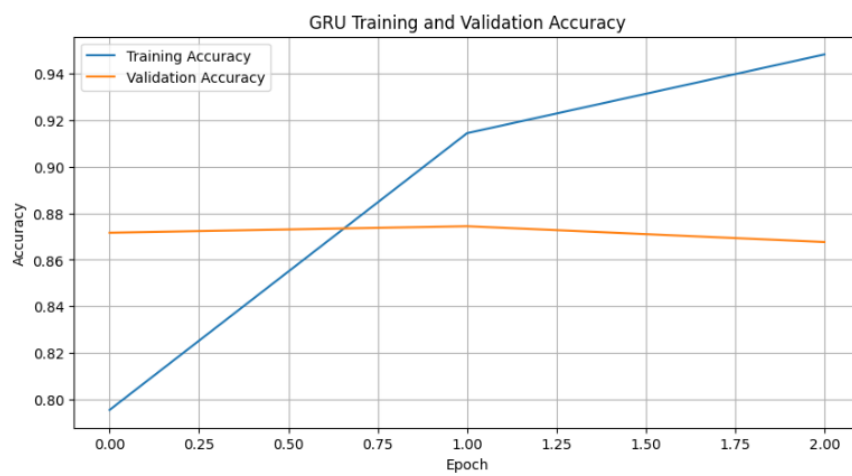
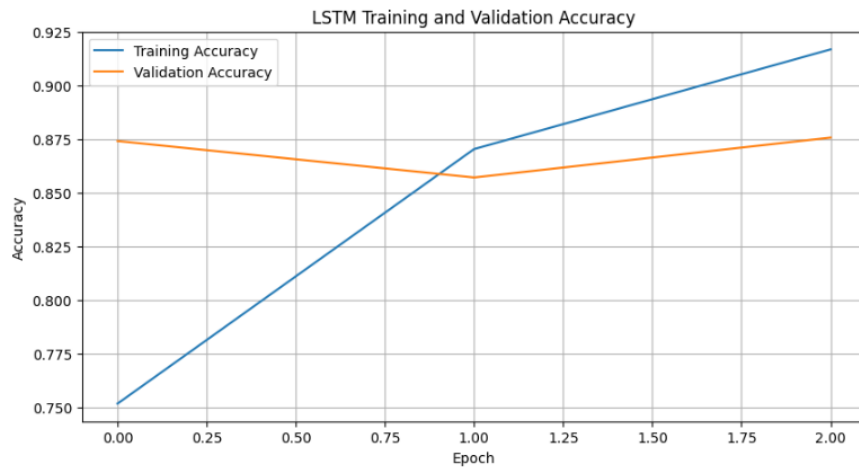
    callbacks=[
        early_stopping
    ],

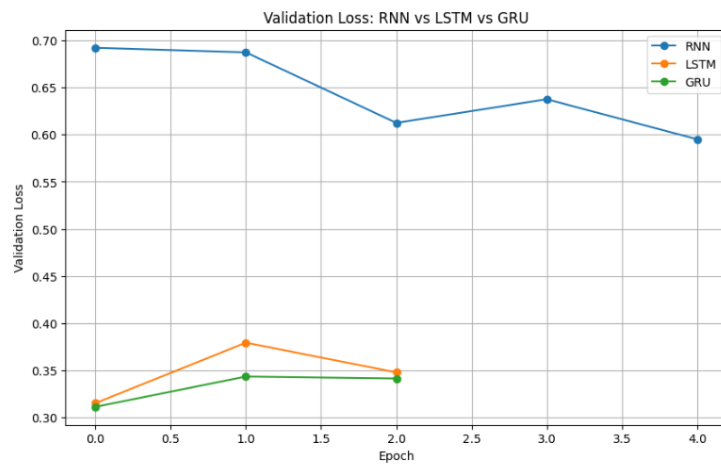
    verbose=1
)

rnn_training_time = time.time() - start_time

print(
    f"\nRNN training time: "
    f"{rnn_training_time:.2f} seconds"
)
```







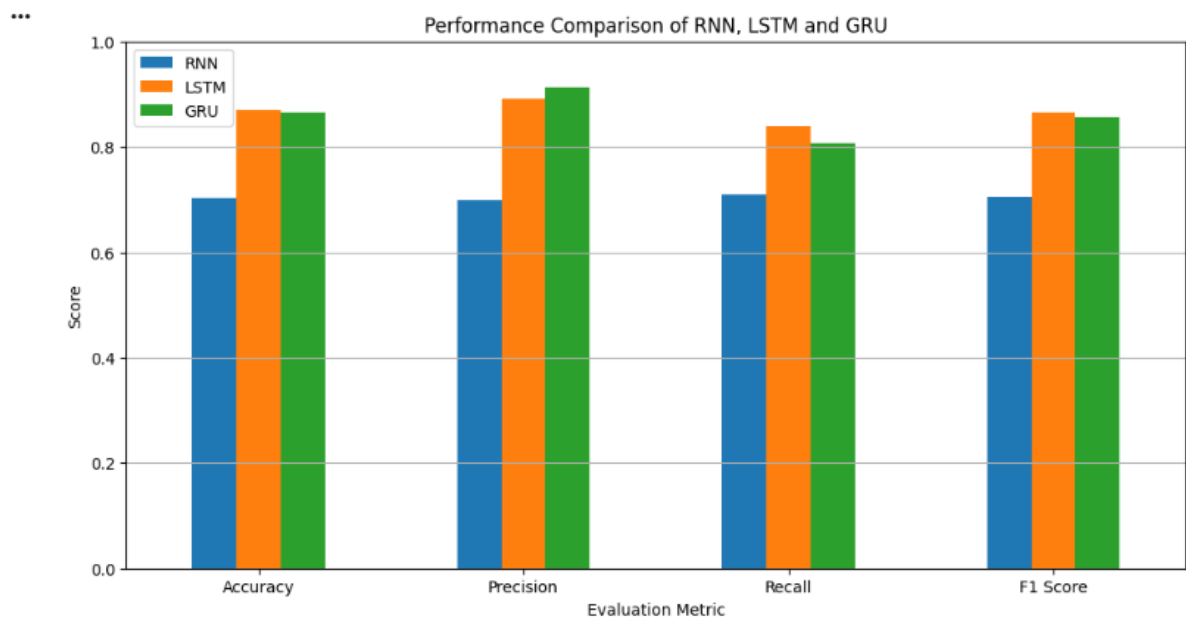
```

results = pd.DataFrame({
    "RNN": rnn_metrics,
    "LSTM": lstm_metrics,
    "GRU": gru_metrics
})

```

results

	RNN	LSTM	GRU
Accuracy	0.701800	0.869000	0.865000
Precision	0.698700	0.892053	0.913083
Recall	0.709600	0.839600	0.806800
F1 Score	0.704108	0.865032	0.856657



RNN Classification Report

	precision	recall	f1-score	support
Negative	0.70	0.69	0.70	2500
Positive	0.70	0.71	0.70	2500
accuracy			0.70	5000
macro avg	0.70	0.70	0.70	5000
weighted avg	0.70	0.70	0.70	5000

*** LSTM Classification Report

	precision	recall	f1-score	support
Negative	0.85	0.90	0.87	2500
Positive	0.89	0.84	0.87	2500
accuracy			0.87	5000
macro avg	0.87	0.87	0.87	5000
weighted avg	0.87	0.87	0.87	5000

*** GRU Classification Report

	precision	recall	f1-score	support
Negative	0.83	0.92	0.87	2500
Positive	0.91	0.81	0.86	2500
accuracy			0.86	5000
macro avg	0.87	0.86	0.86	5000
weighted avg	0.87	0.86	0.86	5000

Review:

This movie was absolutely fantastic.
The acting was excellent and the story was very engaging.
I really enjoyed watching it.

RNN:

('Positive', np.float32(0.78241473))

LSTM:

('Positive', np.float32(0.8955839))

GRU:

('Positive', np.float32(0.9558731))

*** Review:

This movie was extremely boring and disappointing.
The story was weak and the acting was terrible.
I did not enjoy watching it.

RNN:

('Positive', np.float32(0.6837904))

LSTM:

('Negative', np.float32(0.025310172))

GRU:

('Negative', np.float32(0.011429567))

	Model	Accuracy	Precision	Recall	F1 Score	Parameters	Training Time (sec)	
0	RNN	0.7018	0.698700	0.7096	0.704108	2601217	66.910403	
1	LSTM	0.8690	0.892053	0.8396	0.865032	2699905	36.326509	
2	GRU	0.8650	0.913083	0.8068	0.856657	2667393	31.645463	